

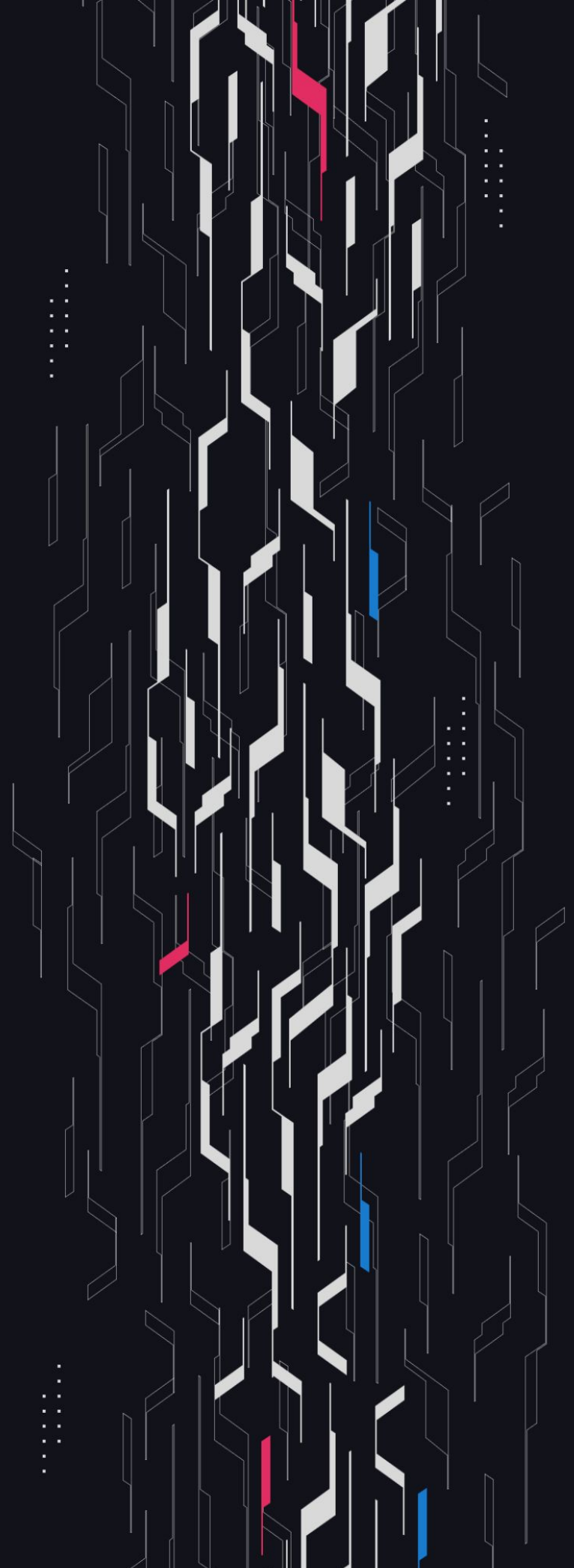
GA GUARDIAN

M0

PYUSDx

Security Assessment

May 24th, 2026



Summary

Audit Firm Guardian

Prepared By CuriousApple, Cryptic Defense

Client Firm M0

Final Report Date May 24, 2026

Audit Summary

M0 engaged Guardian to review the PYUSDx codebase. From April 20, 2026 to May 22, 2026, a team of 2 auditors reviewed the source code and recorded the findings in this report.

Confidence Ranking

Given the lack of High and Critical issues detected during the main review, Guardian assigns a Confidence Ranking of 5 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

✓ Verify the authenticity of this report on Guardian's GitHub: <https://github.com/guardianaudits>

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 6

Smart Contract Risk Assessment

Findings & Resolutions 9

Addendum

Disclaimer 32

About Guardian 33

Project Overview

Project Summary

Project Name	PYUSDX
Language	Solidity
Codebase	https://github.com/m0-foundation/PYUSDX
Commit(s)	Main Review: f56dccf2db25c0be29775e5e871d87da8c0b5996 Remediation Review: 876c90fc94dc02a757cf700e2d010f7e1fb465e7

Audit Summary

Delivery Date	May 24, 2026
Audit Methodology	Static Analysis, Manual Review

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
● Critical	0	0	0	0	0	0
● High	0	0	0	0	0	0
● Medium	0	0	0	0	0	0
● Low	8	0	0	6	0	2
● Info	13	0	0	7	1	5

Audit Scope & Methodology

IPYUSDx.sol
PYUSDx.sol
ReentrancyLock.sol
SwapFacility.sol
Extension.sol
ExtensionFactory.sol
IMinterGateway.sol
MinterGateway.sol
RateLimiter.sol
MultiMint.sol
YieldToOne.sol
HubPortal.sol
Portal.sol
SpokePortal.sol
ReentrancyLock.sol
BytesParser.sol
PayloadEncoder.sol
TypeConverter.sol
BridgeAdapter.sol
MigrateArbitrumSpokePortal.s.sol

MigrateBaseSpokePortal.s.sol
MigrateHubPortal.s.sol
MigrateHubPortalBase.sol
MigrateOptimismSpokePortal.s.sol
MigratePortalBase.sol
MigrateSpokePortalBase.sol
LayerZeroBridgeAdapter.sol
HubPortalV1StorageCleaner.sol
PortalV1StorageCleaner.sol
SpokePortalV1StorageCleaner.sol
IHubPortalV1.sol
IPortalV1.sol
ISpokePortalV1.sol
HyperlaneBridgeAdapter.sol
WormholeBridgeAdapter.sol
ExecutorOptions.sol
OptionsBuilder.sol
StandardHookMetadata.sol
ExecutorMessages.sol
RelayInstructions.sol

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: <i>High</i>	Impact: <i>Medium</i>	Impact: <i>Low</i>
Likelihood: <i>High</i>	● Critical	● High	● Medium
Likelihood: <i>Medium</i>	● High	● Medium	● Low
Likelihood: <i>Low</i>	● Medium	● Low	● Low

Impact

- High** Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.
- Medium** A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.
- Low** Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

- High** The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.
- Medium** An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.
- Low** Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.
Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Findings & Resolutions

ID	Title	Category	Severity	Status
L-01	IssuerGateway Does Not Snapshot Proposal Timing	Validation	● Low	Resolved
L-02	Capital-efficient Bridge DoS	Gaming	● Low	Acknowledged
L-03	Rate Limiters: Dead Code	Best Practices	● Low	Resolved
L-04	Fee Lost On Freeze Despite Safe Transfer Path	Rewards	● Low	Acknowledged
L-05	Wormhole Routes Have No On-Chain Fee Quoting	Compatibility	● Low	Acknowledged
L-06	Portal Incompatible With Wormhole Adapter	Compatibility	● Low	Acknowledged
L-07	Delegate Clearance On Role Revocation	Unexpected Behavior	● Low	Acknowledged
L-08	replaceAsset() And Low-decimal Assets	Rounding	● Low	Acknowledged
I-01	MultiMint Assumes 1:1 Stablecoin Parity	Trust Assumptions	● Info	Partially Resolved
I-02	Redemption Is Fully Custodial	Documentation	● Info	Acknowledged
I-03	Misleading YieldClaimed Event On skipTransfer	Events	● Info	Acknowledged
I-04	No Cleanup For Expired Mint Proposals	Best Practices	● Info	Acknowledged
I-05	Portal And EarnerManager: RateLimiter	Informational	● Info	Resolved

Findings & Resolutions

ID	Title	Category	Severity	Status
I-06	Per-account Earner Rates And Total Principal	Informational	● Info	Acknowledged
I-07	Rate Limiter Provides Limited Protection	Informational	● Info	Acknowledged
I-08	replaceAsset Lacks Asset Registration Check	Validation	● Info	Resolved
I-09	Burn Lacks Pre-Claim For Earning Accounts	Suggestion	● Info	Resolved
I-10	ETH Sent To Bridge Adapters Is Not Recoverable	Validation	● Info	Acknowledged
I-11	Wormhole Consistency Level Undefined	Warning	● Info	Acknowledged
I-12	Missing AccessControl Initializer Call	Best Practices	● Info	Resolved
I-13	Repeated Wraps Of High-decimal Assets And Dust	Rounding	● Info	Resolved

L-01 | IssuerGateway Does Not Snapshot Proposal Timing

Category	Severity	Location	Status
Validation	● Low	IssuerGateway.sol:114-119	Resolved

Description

`proposeMint` computes `activeAt` and `expiresAt` at proposal time and emits those values in the `MintProposed` event, but the protocol stores only `createdAt` in `MintProposal`. The proposal itself does not retain the timing parameters that were in effect when it was created.

Instead, both `cancelMint` and `mint` recompute the proposal's validity window at execution time using the current global values of `mintDelay` and `mintTTL`:

```
// cancelMint - reads current $.mintDelay
activeAt = proposal.createdAt + $.mintDelay;
// mint - reads current $.mintDelay and $.mintTTL
activeAt  = proposal.createdAt + $.mintDelay;
expiresAt = activeAt          + $.mintTTL;
```

Because `setMintDelay` and `setMintTTL` take effect immediately and are not snapshotted per proposal, any admin update retroactively changes the execution window for all pending proposals.

This creates a mismatch between what was announced in `MintProposed` and what is ultimately enforced on-chain. A proposal that appeared scheduled to become valid at one time, or expire at one time, can later become executable earlier, later, or expire sooner or later than originally signaled. In other words, proposal timing is not immutable once proposed; it remains subject to governance or admin intervention until execution or cancellation.

Recommendation

Snapshot `activeAt` and `expiresAt` into the `MintProposal` struct at creation time and use the stored values in `cancelMint` and `mint`:

Resolution

M0 Team: The issue was resolved in PR#52.

L-02 | Capital-efficient Bridge DoS

Category	Severity	Location	Status
Gaming	● Low	RateLimiter.sol:21, RateLimiter.sol:128-133, Portal.sol:197	Acknowledged

Description

The rate limit bucket is keyed by issuer address (`mapping(address issuer => Bucket)`), not by sender. Portal is the sole `ISSUER_ROLE` holder for bridge-triggered mints, meaning all cross-chain recipients on a given chain share a single bucket. An attacker can drain the entire bucket in a single transaction:

- Bridge capacity PYUSDx from chain A → B — Portal-B’s bucket is fully consumed
- All subsequent `receiveMessage` calls on chain B revert with `RateLimitExceeded` until the bucket refills
- Bridge capacity PYUSDx back from B → A — Portal-A’s bucket is consumed
- Repeat indefinitely

Cost to attacker: Only LayerZero (LZ) gas fees (~\$5–50 per round trip, depending on the chain). The PYUSDx capacity is recovered on the destination chain with each hop—resulting in zero net capital loss.

Impact:

Legitimate cross-chain transfers are blocked for the duration of the bucket refill window (`capacity / refillPerSecond` seconds). Sustaining the attack requires only enough ETH to cover LZ fees.

Recommendation

Consider a tiered bridge fee model on `sendToken`:

Below threshold X: zero protocol fee. Gas and LZ relayer costs already represent a meaningful fraction of small transfer values, making low-value bucket drain attempts self-detering.

At or above X: charge Y basis points on the full bridged amount. At this scale, fixed gas/LZ costs are negligible relative to capital deployed, and the fee is the only economic deterrent.

Resolution

M0 Team: Acknowledged.

L-03 | Rate Limiters: Dead Code

Category	Severity	Location	Status
Best Practices	● Low	RateLimiter.sol:175-178	Resolved

Description

`_calculateRemainingAmount` uses `Math.tryMul` and `Math.tryAdd` to guard against overflow, returning `capacity` as a fallback if either operation overflows:

```
(bool mulOk, uint256 refillAmount) = Math.tryMul(elapsed, refillPerSecond);
(bool addOk, uint256 total)         = Math.tryAdd(remaining, refillAmount);
return (!mulOk || !addOk) ? capacity : uint128(Math.min(total, capacity));
```

However, both overflow checks are statically unreachable given the input types:

- `elapsed` is computed as `uint40 - uint40`, then zero-extended to `uint256`, so its value is bounded by (2^{40}) .
- `refillPerSecond` is `uint128`, bounded by (2^{128}) .
- Product: $(2^{40} \times 2^{128} = 2^{168}) \ll 2^{256}$ → `tryMul` will never return `false`.
- `remaining` is `uint128`, bounded by (2^{128}) .
- Sum: $(2^{128} + 2^{168} < 2^{256})$ → `tryAdd` will never return `false`.

As a result, the branch `(!mulOk || !addOk) ? capacity` is dead code. The function will always execute:

```
uint128(Math.min(total, capacity))
```

Impact:

There is no security risk. However, there are two minor consequences:

- Gas: `Math.tryMul` and `Math.tryAdd` each perform checked arithmetic internally, which is more expensive than direct operations. Since `_enforceRateLimit` is called on every mint, this overhead accumulates across all issuers over time.
- Readability: A reader encountering `tryMul/tryAdd` may reasonably assume that overflow is a real concern. This could lead to unnecessary audit effort or, worse, reliance on the fallback path in future changes where type bounds may differ.

Recommendation

Consider removing the dead code.

Resolution

M0 Team: The issue was resolved in PR#56.

L-04 | Fee Lost On Freeze Despite Safe Transfer Path

Category	Severity	Location	Status
Rewards	● Low	PYUSDX.sol: 458-478	Acknowledged

Description

`_beforeFreeze` calls `_claimFor(account, true)`, which adds yield to the account's balance but skips both net yield to `claim_recipient` and fee to `earner_manager` transfers. The `skipTransfer` flag exists to prevent the freeze from reverting if `claimRecipient` is frozen. However, the fee transfer goes to `earnerManager`, a protocol address that is effectively never frozen, and the contract is not necessarily paused during a freeze. The fee could safely be collected, but the `skipTransfer=true` skips it.

The fee is only recoverable via manual `forceTransfer` from the frozen account before unfreezing, requiring off-chain record-keeping of the fee amount and current `earner_manager` address. If the admin unfreezes without performing this step, the fee is permanently lost.

Recommendation

When not paused, transfer the fee to the `earner_manager` before returning:

```
if (skipTransfer) {
+   if (!paused() && fee > 0) {
+       address feeRecipient = $.earnerManager;
+   }
+   ...
return (yieldWithFee, fee, yieldNetOfFee);
}
```

Resolution

M0 Team: Acknowledged.

L-05 | Wormhole Routes Have No On-Chain Fee Quoting

Category	Severity	Location	Status
Compatibility	● Low	WormholeBridgeAdapter.sol: 81-86	Acknowledged

Description

`Portal::quote()` is used for estimating cross-chain transfer fees, but it reverts for Wormhole routes because `WormholeBridgeAdapter::quote()` unconditionally reverts with `OnChainQuoteNotSupported()`. Users and integrating contracts have no on-chain mechanism to determine the required `msg.value` before calling `sendToken`.

However, an on-chain quoting path does exist. Wormhole's `ExecutorQuoter::requestQuote` can estimate the executor fee, and `ICoreBridge.messageFee()` provides the core bridge fee, but neither is integrated into the adapter's quote function. If a user underpays, the Executor silently accepts the request (due to the `payable(payeeAddress).transfer(msg.value);` line in `Executor.sol`), but the relay provider never delivers the message, and tokens are already burned/locked on the source chain.

Recommendation

Integrate Wormhole's on-chain quote resolution (<https://wormhole.com/docs/protocol/infrastructure/relayers/executor-framework/#on-chain-quote-resolution>) via `ExecutorQuoterRouter`. This was designed for integrators that cannot pass off-chain signatures, and must rely on `Portal.quote()`. Store the `ExecutorQuoterRouter` address and call `quoteExecution()` to derive fees on-chain, combining the result with `ICoreBridge.messageFee()` to return the fee estimate.

Resolution

M0 Team: Acknowledged.

L-06 | Portal Incompatible With Wormhole Adapter

Category	Severity	Location	Status
Compatibility	● Low	Portal.sol: 385	Acknowledged

Description

The documentation states "Other adapters (Hyperlane, Wormhole, etc.) can be added without modifying Portal", however that currently is not the case for a standard Wormhole adapter. The PYUSD Portal's `_sendMessage` passes four parameters to `IBridgeAdapter::sendMessage` (destination chain, gas limit, refund address, and payload). There is no mechanism to pass additional arguments. The M0 Portal supports a fifth `extraArguments` parameter used by the Wormhole adapter to receive the off-chain `signedQuote` required by the Executor framework. Without this parameter, the PYUSD Portal cannot integrate with a Wormhole adapter without either modifying the Portal interface (upgrading) or designing a non-standard adapter that obtains the signed quote through an alternative mechanism.

Recommendation

Add an optional `bytes calldata extraArguments` parameter to the Portal's `_sendMessage` and the `IBridgeAdapter.sendMessage` interface, matching the M0 Portal design.

Resolution

M0 Team: Acknowledged.

L-07 | Delegate Clearance On Role Revocation

Category	Severity	Location	Status
Unexpected Behavior	● Low	LayerZeroBridgeAdapter.sol: 99-100	Acknowledged

Description

Revoking the `OPERATOR_ROLE` in `LayerZeroBridgeAdapter` automatically clears the LayerZero endpoint delegate by setting it to `address(0)`. This creates a availability risk: it removes the ability to call privileged recovery functions (`skip`, `clear`, `nilify`) during a nonce-clog scenario, potentially leading to persistent denial of service.

- When an operator is revoked, the contract automatically clears the delegate.
- LayerZero endpoint functions (`skip`, `clear`, `nilify`) can only be called by:
 - The OApp itself, or
 - Its registered delegate
 - The adapter does not expose wrapper functions for these operations.

Example Scenario: Nonce-Clog DoS

Recommendation

Make sure that the delegate is reset to appropriate address via `setDelegate` after its clearance.

Resolution

M0 Team: Acknowledged.

L-08 | replaceAsset() And Low-decimal Assets

Category	Severity	Location	Status
Rounding	● Low	MultiMint.sol: 296	Acknowledged

Description

MultiMint.totalAssets() is tracked in 6-decimal extension/PYUSDx units, but replaceAsset() removes reserve assets in the asset's native decimals.

For assets with fewer than 6 decimals, replaceAsset() converts the requested PYUSDx amount into assetAmount using truncation, then:

- decreases assetBalance by truncated assetAmount
- decreases totalAssets by the full untruncated amount

This can make totalAssets smaller than the normalized value of actual remaining reserves.

Affected logic:

- wrap path adds normalized amount: src/platform/projects/MultiMint.sol:260
- replace path subtracts raw amount: src/platform/projects/MultiMint.sol:279

Example

With a 2-decimal asset:

- wrap 10171 asset units
- normalized backing added = 10171 * 10000 = 101,710,000

Then call replaceAsset(..., amount = 10,171):

- 10,171 PYUSDx units converts to 1 asset unit after truncation
- actual normalized reserve removed = 10,000
- but contract subtracts 10,171 from totalAssets

Post-state:

- expected normalized reserves = 101,700,000
- stored totalAssets = 101,699,829

This creates accounting drift and can silently overcharge replacers.

Impact

- totalAssets can become inconsistent with actual normalized reserve balances
- users can overpay when replacing into low-decimal assets

Recommendation

Consider only accepting the the normalized amount for PYUSDx, and refund the rest via Swap Facility. In above example, 10000 would be transferred to extension while 171 is refunded back.

Resolution

M0 Team: Acknowledged.

I-01 | MultiMint Assumes 1:1 Stablecoin Parity

Category	Severity	Location	Status
Trust Assumptions	● Info	Multimint.sol	Resolved

Description

MultiMint converts between alt-asset amounts and extension-token amounts using only decimal scaling via `_convertAmounts`, with no price oracle. As a result, a deposit of 1 USDC always mints exactly 1 PYUSD_X-worth of extension tokens, and `replaceAsset(1 PYUSDX)` always returns exactly 1 USDC, regardless of market price.

Behavior when the asset depegs below \$1

In a depeg scenario, `wrap(USDC, ...)` becomes immediately exploitable. A user can deposit an asset worth only \$0.87 and still receive 1e6 extension tokens, which remain redeemable through `unwrap` for \$1 worth of PYUSD_X. This creates instant arbitrage at the expense of the extension’s existing PYUSD_X backing pool. Each such wrap reduces `pyusdxBacking` per extension token and dilutes existing holders.

At the same time, `replaceAsset` becomes economically irrational: no rational actor will spend \$1 of PYUSD_X to receive only \$0.87 of USDC. The result is that reserves accumulate the depegged asset, while no one is incentivized to rotate it out. Net effect: the reserve composition deteriorates as it fills with below-peg assets; extension token holders become collectively undercollateralized; and redemptions become a race. Early unwrappers extract full PYUSD_X value, while late holders are left with tokens backed only by impaired assets after `pyusdxBacking` has been exhausted.

Behavior when the asset trades above \$1 If the asset appreciates above peg, the incentives reverse. `replaceAsset` becomes attractive because a user can pay \$1 of PYUSD_X and receive \$1.05 of USDC. This drains high-quality reserves until either caps block further inflows or the appreciated asset is fully extracted.

Meanwhile, `wrap(USDC, ...)` becomes unattractive: the user contributes \$1.05 of value but still receives only 1e6 extension tokens, redeemable for just \$1. Rational users stop wrapping, which reduces the inflow of good collateral. Net effect: above-peg assets are arbitrated out of reserves, leaving behind only at-peg or below-peg assets.

Recommendation

Beware of this trust assumption.

Resolution

M0 Team: The issue was resolved in PR#62.

I-02 | Redemption Is Fully Custodial

Category	Severity	Location	Status
Documentation	● Info	IssuerGateway.sol:183-189	Acknowledged

Description

IssuerGateway.burn is restricted by onlyRole(OPERATOR_ROLE) and burns tokens exclusively from msg.sender’s balance. As implemented, there is no function that allows a PYUSD holder to directly burn their own tokens on-chain.

Redemption Flow

The only available redemption path is effectively mediated:

1. A holder enters into a bilateral agreement with an authorized issuer entity (holder of ISSUER_ROLE).
2. The holder transfers PYUSD to the issuer (off-chain or via an agreed transfer mechanism).
3. An operator calls IssuerGateway.burn(amount) to burn the received tokens from their own balance.
4. The issuer settles the corresponding PYUSD/USD value with the holder through off-chain means.

Recommendation

Consider documenting the custodial redemption dependency explicitly in user-facing materials and the protocol spec

Resolution

M0 Team: Acknowledged.

I-03 | Misleading YieldClaimed Event On skipTransfer

Category	Severity	Location	Status
Events	● Info	PYUSDx.sol: 446-462	Acknowledged

Description

When `_claimFor` is called with `skipTransfer=true` (during `_beforeFreeze` and `paused` `_setAccountInfo`), the `YieldClaimed(account, yieldNetOfFee)` event is emitted before the early return. Although the account actually receives `yieldWithFee`, this event is misleading since no fee is collected by the earner manager, and no yield is actually routed to `claimRecipient`. Off-chain systems indexing `YieldClaimed` could incorrectly assume the fee was paid and yield was distributed.

Recommendation

Consider emitting a separate event for yield handling when `skipTransfer=true` or move the `YieldClaimed` emission to after the routing logic.

Resolution

M0 Team: Acknowledged.

I-04 | No Cleanup For Expired Mint Proposals

Category	Severity	Location	Status
Best Practices	● Info	IssuerGateway.sol: 137	Acknowledged

Description

Expired mint proposals cannot be cancelled (`ActiveMintProposal` revert) or executed (`ExpiredMintProposal` revert), leaving the storage permanently occupied. Over time, frequently expiring proposals accumulate dead storage with no cleanup mechanism.

Recommendation

Consider adding a permissionless function for clearing expired proposals:

```
function clearExpiredProposal(uint48 mintId) external {
  IssuerGatewayStorage storage \$_ = _getIssuerGatewayStorage();
  MintProposal storage proposal = \$.mintProposals[mintId];
  ...
  delete \$.mintProposals[mintId];
}
```

Resolution

M0 Team: Acknowledged.

I-05 | Portal And EarnerManager: RateLimiter

Category	Severity	Location	Status
Informational	● Info	RateLimiter.sol:128-133 PYUSDx.sol:116-118 PYUSDx.sol:136-140 Portal.sol:197	Resolved

Description

PYUSDx has two mint paths that bypass IssuerGateway's timelock.
Caller	Function	Gating
ISSUER_ROLE holder (Portal)	PYUSDx.mint → _enforceRateLimit(msg.sender)	Rate limit bucket only
earnerManager	PYUSDx.distributeReward → _enforceRateLimit(msg.sender)	Rate limit bucket only
Both paths converge on _enforceRateLimit, which includes an explicit bypass for unconfigured callers:
if (bucket.lastRefillTime == 0) return; // unconfigured → unlimited

Recommendation

Consider configuring rate limits for both portal and earner manager

Resolution

M0 Team: The issue was resolved in PR#51.

I-06 | Per-account Earner Rates And Total Principal

Category	Severity	Location	Status
Informational	● Info	PYUSDx.sol:36 PYUSDx.sol:515–523 PYUSDx.sol:541–551	Acknowledged

Description

M0 model (single global index):

```
principal_removed = amount / idx
principal_added   = amount / idx
net Δ principal   = 0    (conserved)
```

PYUSDx model (per-account index):

```
principal_removed = ceil(amount / idx_A)
principal_added   = floor(amount / idx_B)
net Δ principal   = floor(amount/idx_B) - ceil(amount/idx_A)  (≠ 0)
```

Transfers between earners with different rates produce asymmetric principal changes due to:

- Different indexes (`idx_A ≠ idx_B`)
- Opposite rounding directions (ceil vs floor)

Example

- Sender A: `idx_A = 2e12` → removes 50 principal
- Recipient B: `idx_B = 4e12` → adds 25 principal
- Net: -25 principal destroyed

Implications

- Path-dependent yield: Moving funds to higher-rate accounts increases total yield over time (extractable via routing strategies)
- No global invariant: `totalEarningPrincipal` is not meaningful without normalization

Recommendation

Beware of this behavior and document it

Resolution

M0 Team: Acknowledged.

I-07 | Rate Limiter Provides Limited Protection

Category	Severity	Location	Status
Informational	● Info	src/PYUSDx.sol, src/abstract/RateLimiter.sol	Acknowledged

Description

The protocol uses a token-bucket rate limiter on minting (`_mint` → `_enforceRateLimit`), which limits damage from a compromised `ISSUER_ROLE`. However, this protection applies only to one path. Several other roles have equal or greater impact with no similar safeguards:

- `RATE_LIMIT_MANAGER_ROLE` – Can remove all limits → enables unlimited minting
- `FREEZE_MANAGER_ROLE` + `FORCED_TRANSFER_MANAGER_ROLE` – Can freeze and fully drain any account instantly
- `PAUSER_ROLE` – Can halt all protocol activity
- `earnerManager` – Can redirect all future yield to attacker
- `DEFAULT_ADMIN_ROLE` – Full takeover: remove limits → grant `ISSUER_ROLE` → mint unlimited (3 tx)

As a result, the rate limiter only helps in a narrow scenario where only the `ISSUER_ROLE` is compromised.

Recommendation

Do not treat the rate limiter as broad defense-in-depth—it protects only one attack path. Consider placing high-privilege roles behind stricter controls (e.g., timelock or higher-threshold multisig) to make attacks observable and interruptible.

Resolution

M0 Team: Acknowledged.

I-08 | replaceAsset Lacks Asset Registration Check

Category	Severity	Location	Status
Validation	● Info	MultiMint.sol: 279	Resolved

Description

MultiMint::_replaceAsset does not verify that the asset was previously registered via setAssetCap. For an unregistered asset, assetDecimals returns 0, causing _fromExtensionToAssetAmount to produce an incorrect value via _convertAmounts(6, 0, amount). The function then reverts at _revertIfInsufficientAssetBacking, since the balance is zero, but the revert reason is misleading because the function lacks an explicit check.

Note _revertIfInvalidAsset only checks if asset is a non-zero address and not pyusdx, it does not check if the asset has already been registered.

Recommendation

Add a registration check early in _replaceAsset:

```
if (assetDecimals(asset) == 0) revert InvalidAsset(asset);
```

Resolution

M0 Team: The issue was resolved in PR#58.

I-09 | Burn Lacks Pre-Claim For Earning Accounts

Category	Severity	Location	Status
Suggestion	● Info	PYUSDX.sol: 121	Resolved

Description

`_beforeFreeze` and `_setAccountInfo` both call `_claimFor` before mutating an earner's balance, but `burn` does not. Currently, this has no impact since `ISSUER_ROLE` is held by `IssuerGateway` (which burns from the operator's own non-earning balance) and `Portal` (which burns from its own non-earning balance). However, the code handles the earning path without claiming first, creating an inconsistency. If `ISSUER_ROLE` is ever granted to a contract that burns directly from earning accounts, unclaimed yield would be subject to rounding loss.

Recommendation

Consider adding `_claimFor(account)` before `_subtractEarningAmount` in `burn` if the account is an earner, document that burn targets are expected to be non-earning accounts.

Resolution

M0 Team: The issue was resolved in PR#61.

I-10 | ETH Sent To Bridge Adapters Is Not Recoverable

Category	Severity	Location	Status
Validation	● Info	HyperlaneBridgeAdapter.sol: 69	Acknowledged

Description

`HyperlaneBridgeAdapter::handle` and `WormholeBridgeAdapter::executeVAAv1` are both payable (required by their respective interfaces) but neither forwards `msg.value` to Portal nor has a withdrawal mechanism. Any ETH sent with these calls is permanently stuck in the adapter contract. While relayers typically send 0 ETH for message-only deliveries, there is no protection against any ETH inclusion (i.e., accidental), and no admin sweep function exists to recover stuck funds.

For example, we can see that Hyperlane's `process` function sends `msg.value` when executing `handle`:

<https://github.com/hyperlane-xyz/hyperlane-monorepo/blob/main/solidity/contracts/Mailbox.sol#L241>

```
// Deliver the message to the recipient.
IMessageRecipient(recipient).handle{value: msg.value}(
  _message.origin(),
  ...
);
```

Recommendation

Consider adding an admin-gated sweep function to recover stuck ETH.

Resolution

M0 Team: Acknowledged.

I-11 | Wormhole Consistency Level Undefined

Category	Severity	Location	Status
Warning	● Info	WormholeBridgeAdapter.sol: 57-72	Acknowledged

Description

`WormholeBridgeAdapter::consistencyLevel` is an immutable set at construction, and currently its value is undefined throughout the repo. Tests use `15`, which is a custom finality level and is not the recommended finalized level. Wormhole's EVM convention defines `1` as finalized, `200` as instant, and `201` as safe. Deploying with a value other than `1` (finalized) introduces reorg risk, and tokens could be minted on the destination chain while the source transaction is rolled back during a chain reorg.

Recommendation

Document the intended production `consistencyLevel` and verify at deployment that it is set to `1` (finalized) for mainnet, or explicitly acknowledge the reorg risk if a lower finality level is chosen (i.e., for speed).

Resolution

M0 Team: Acknowledged.

I-12 | Missing AccessControl Initializer Call

Category	Severity	Location	Status
Best Practices	● Info	BridgeAdapter.sol: 52	Resolved

Description

`Portal::initialize` and `BridgeAdapter::initialize` skip calling `__AccessControl_init()` before granting roles, while `ExtensionFactory`, `ExtensionBeacon`, and `IssuerGateway` all call it. In current OpenZeppelin versions this is a no-op, so there is no impact. However, the inconsistency means a future OpenZeppelin upgrade that adds logic to `__AccessControl_init()` would only take effect in some contracts and silently be missed in others.

Recommendation

Add `__AccessControl_init()` to `Portal::initialize` and `BridgeAdapter::_initialize`.

Resolution

M0 Team: The issue was resolved in PR#63.

I-13 | Repeated Wraps Of High-decimal Assets And Dust

Category	Severity	Location	Status
Rounding	● Info	MultiMint.sol: 243	Resolved

Description

The invariant $\text{totalAssets} == \text{sum}(\text{assetBalance} * \text{factor})$ may not hold true for high decimal assets (>6) as well.

Examples

8-decimal asset:

- wrap 1 leaves remainder 77
- wrap 2 leaves remainder 91
- combined dust 168 crosses the /100 boundary once
- result: aggregate-normalized reserves exceed stored accounting by 1

18-decimal asset:

- wrap 1 leaves remainder 493,295,770,473
- wrap 2 leaves remainder 793,244,671,737
- combined dust exceeds $1e12$
- result: aggregate-normalized reserves exceed stored accounting by 1

Recommendation

Beware of this consideration.

Resolution

M0 Team: The issue was resolved in PR#60.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit <https://guardianaudits.com>

To view our audit portfolio, visit <https://github.com/guardianaudits>

To book an audit, message <https://t.me/guardianaudits>